

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and
Commerce Department of Information Technology (B.Sc.I.T Semester V)
Applied Business Analytics

Practical 6

Roll No.: T050	Name: Aashu Yadav
Class: TYIT	Batch: 1
Date of Assignment: 2/09/26	Date/Time of Submission: 2/09/26

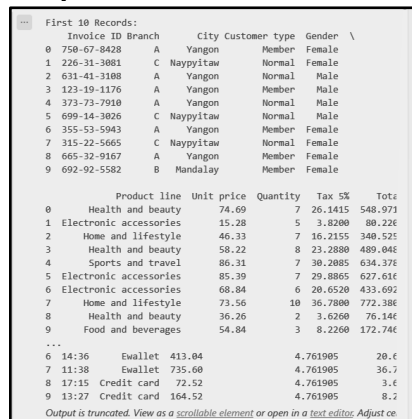
Aim: Decision Making under Uncertainty.
Part A: Develop a Decision Tree Model

1. Load the supermarket_sales.csv dataset into a Pandas DataFrame and display the first 10 records.

Code:

```
# Import required libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
df = pd.read_csv("supermarket_sales.csv")
print("First 10 Records:")
print(df.head(10))
```

Output:



First 10 Records:

	Invoice ID	Branch	City	Customer type	Gender	
0	750-67-8428	A	Yangon	Member	Female	
1	226-31-3881	C	Naypyitaw	Normal	Female	
2	631-41-3108	A	Yangon	Normal	Male	
3	123-19-1176	A	Yangon	Member	Male	
4	373-73-7918	A	Yangon	Normal	Male	
5	699-14-3826	C	Naypyitaw	Normal	Male	
6	355-53-5943	A	Yangon	Member	Female	
7	315-22-5665	C	Naypyitaw	Normal	Female	
8	665-32-9167	A	Yangon	Member	Female	
9	692-92-5582	B	Mandalay	Member	Female	

	Product line	Unit price	Quantity	Tax 5%	Total
0	Health and beauty	74.60	7	26.1415	548.971
1	Electronic accessories	15.28	5	3.8200	88.228
2	Home and lifestyle	46.33	7	16.2155	348.525
3	Health and beauty	58.22	8	23.2880	489.048
4	Sports and travel	86.31	7	30.2085	634.378
5	Electronic accessories	85.39	7	29.3865	627.618
6	Electronic accessories	68.84	6	20.6520	433.692
7	Home and lifestyle	73.56	10	36.7800	772.388
8	Health and beauty	36.26	2	3.6260	76.148
9	Food and beverages	54.84	3	8.2260	172.748

Output is truncated. View as a scrollable element or open in a text editor. Adjust co...

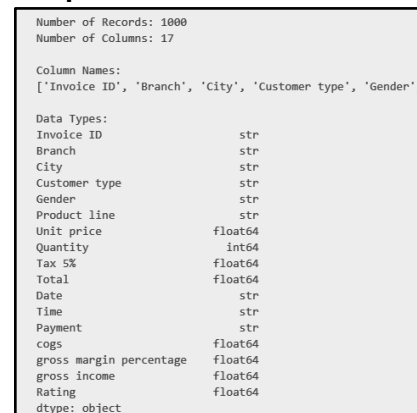
2. Display the number of records, number of columns, column names, and data types of the dataset.

Code:

```
print("\nNumber of Records:", df.shape[0])
print("Number of Columns:", df.shape[1])
print("\nColumn Names:")
print(df.columns.tolist())
print("\nData Types:")
```

```
print(df.dtypes)
```

Output:



Number of Records: 1000
Number of Columns: 17

Column Names:
['Invoice ID', 'Branch', 'City', 'Customer type', 'Gender',
Product line', 'Unit price', 'Quantity', 'Tax 5%', 'Total', 'Date', 'Time', 'Payment', 'cogs', 'gross margin percentage', 'gross income', 'Rating']

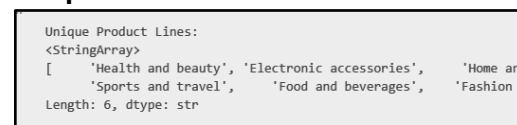
Data Types:
Invoice ID str
Branch str
City str
Customer type str
Gender str
Product line str
Unit price float64
Quantity int64
Tax 5% float64
Total float64
Date str
Time str
Payment str
cogs float64
gross margin percentage float64
gross income float64
Rating float64
dtype: object

3. Display all unique values of the product_line column.

Code:

```
print("\nUnique Product Lines:")
print(df["Product line"].unique())
```

Output:



Unique Product Lines:
<StringArray>
['Health and beauty', 'Electronic accessories', 'Home and lifestyle', 'Sports and travel', 'Food and beverages', 'Fashion']
Length: 6, dtype: str

4. Calculate the total revenue generated by each product line.

Code:

```
total_revenue = df.groupby("Product line")["Total"].sum()
print("\nTotal Revenue by Product Line:")
print(total_revenue)
```

Output:

Aashu Yadav
T050

```
Total Revenue by Product Line:
Product line
Electronic accessories    54337.5315
Fashion accessories       54305.8950
Food and beverages       56144.8440
Health and beauty        49193.7390
Home and lifestyle        53861.9130
Sports and travel         55122.8265
Name: Total, dtype: float64
```

5. Calculate the average revenue generated by each product line.

Code:

```
average_revenue = df.groupby("Product line")["Total"].mean()
print("\nAverage Revenue by Product Line:")
print(average_revenue)
```

Output:

```
Average Revenue by Product Line:
Product line
Electronic accessories    319.632538
Fashion accessories       305.089298
Food and beverages       322.671517
Health and beauty        323.643020
Home and lifestyle        336.636956
Sports and travel         332.065220
Name: Total, dtype: float64
```

6. Calculate the total quantity sold for each product line.

Code:

```
total_quantity = df.groupby("Product line")["Quantity"].sum()
print("\nTotal Quantity Sold by Product Line:")
print(total_quantity)
```

Output:

```
Total Quantity Sold by Product Line:
Product line
Electronic accessories    971
Fashion accessories       902
Food and beverages       952
Health and beauty        854
Home and lifestyle        911
Sports and travel         920
Name: Quantity, dtype: int64
```

7. Calculate the median value of the revenue column.

Code:

```
median_revenue = df["Total"].median()
print("\nMedian Revenue:", median_revenue)
```

Output:

```
Median Revenue: 253.848
```

8. Classify every transaction as High Revenue or Low Revenue using the median revenue as the decision threshold.

9. Create a new column named revenue_level to store the High Revenue and Low

Revenue classifications.

Code:

```
df["revenue_level"] = np.where(
    df["Total"] >= median_revenue,
    "High Revenue",
    "Low Revenue"
)
```

```
print("\nRevenue Classification:")
print(df[["Product line", "Total", "revenue_level"]].head(10))
```

Output:

```
Revenue Classification:
Product line  Total revenue_level
0  Health and beauty  548.9715  High Revenue
1  Electronic accessories  80.2200  Low Revenue
2  Home and lifestyle  340.5255  High Revenue
3  Health and beauty  489.0480  High Revenue
4  Sports and travel  634.3785  High Revenue
5  Electronic accessories  627.6165  High Revenue
6  Electronic accessories  433.6920  High Revenue
7  Home and lifestyle  772.3800  High Revenue
8  Health and beauty   76.1460  Low Revenue
9  Food and beverages  172.7460  Low Revenue
```

10. Calculate the number of High Revenue and Low Revenue transactions for each product line.

Code:

```
revenue_counts = pd.crosstab(
    df["Product line"],
    df["revenue_level"]
)
print("\nHigh and Low Revenue Transactions:")
print(revenue_counts)
```

Output:

```
High and Low Revenue Transactions:
revenue_level  High Revenue  Low Revenue
Product line
Electronic accessories      79         91
Fashion accessories         84         94
Food and beverages          86         88
Health and beauty           81         71
Home and lifestyle           82         78
Sports and travel           88         78
```

11. Calculate the probability of High Revenue and Low Revenue for each product line.

Code:

```
revenue_probability = pd.crosstab(
    df["Product line"],
    df["revenue_level"],
    normalize="index"
)
print("\nRevenue Probabilities:")
```

Aashu Yadav
T050

```
print(revenue_probability)
```

Output:

Revenue Probabilities:		
revenue_level	High Revenue	Low Revenue
Product line		
Electronic accessories	0.464706	0.535294
Fashion accessories	0.471910	0.528090
Food and beverages	0.494253	0.505747
Health and beauty	0.532895	0.467105
Home and lifestyle	0.512500	0.487500
Sports and travel	0.530120	0.469880

12. Verify that the probabilities of all possible revenue outcomes for each product line add up to 1.

Code:

```
print("\nProbability Sum:")
print(revenue_probability.sum(axis=1))
```

Output:

Probability Sum:	
Product line	
Electronic accessories	1.0
Fashion accessories	1.0
Food and beverages	1.0
Health and beauty	1.0
Home and lifestyle	1.0
Sports and travel	1.0
dtype: float64	

13. Calculate the average revenue for High Revenue and Low Revenue transactions for each product line.

Code:

```
average_payoff = df.groupby(
    ["Product line", "revenue_level"]
)[["Total"]].mean()
print("\nAverage Revenue / Payoff:")
print(average_payoff)
```

Output:

Average Revenue / Payoff:		
Product line	revenue_level	
Electronic accessories	High Revenue	537.006152
	Low Revenue	130.923577
Fashion accessories	High Revenue	512.329375
	Low Revenue	119.896037
Food and beverages	High Revenue	515.127680
	Low Revenue	134.589358
Health and beauty	High Revenue	497.221407
	Low Revenue	125.616972
Home and lifestyle	High Revenue	529.582738
	Low Revenue	133.796519
Sports and travel	High Revenue	518.546438
	Low Revenue	121.676154
Name: Total, dtype: float64		

14. Create a DataFrame containing Product Line, Revenue Level, Probability, and Payoff.

Code:

```
decision_data = []
```

```
for product in df["Product line"].unique():
    for level in ["High Revenue", "Low Revenue"]:
        probability =
revenue_probability.loc[product].get(level, 0)
    if (product, level) in average_payoff.index:
        payoff = average_payoff.loc[(product, level)]
    else:
        payoff = 0
    decision_data.append([
        product,
        level,
        probability,
        payoff
    ])
decision_df = pd.DataFrame(
    decision_data,
    columns=[
        "Product Line",
        "Revenue Level",
        "Probability",
        "Payoff"
    ]
)
print("\nDecision DataFrame:")
print(decision_df)
```

Output:

Decision DataFrame:				
	Product Line	Revenue Level	Probability	Payoff
0	Health and beauty	High Revenue	0.532895	497.221407
1	Health and beauty	Low Revenue	0.467105	125.616972
2	Electronic accessories	High Revenue	0.464706	537.006152
3	Electronic accessories	Low Revenue	0.535294	130.923577
4	Home and lifestyle	High Revenue	0.512500	529.582738
5	Home and lifestyle	Low Revenue	0.487500	133.796515
6	Sports and travel	High Revenue	0.530120	518.546438
7	Sports and travel	Low Revenue	0.469880	121.676154
8	Food and beverages	High Revenue	0.494253	515.127680
9	Food and beverages	Low Revenue	0.505747	134.589358
10	Fashion accessories	High Revenue	0.471910	512.329375
11	Fashion accessories	Low Revenue	0.528090	119.896037

15. Identify the decision alternatives and uncertain outcomes from the DataFrame.

Code:

```
decision_alternatives = df["Product line"].unique()
uncertain_outcomes = [
    "High Revenue",
    "Low Revenue"
]
print("\nDecision Alternatives:")
print(decision_alternatives)
print("\nUncertain Outcomes:")
print(uncertain_outcomes)
```

Output:

Decision Alternatives:		
<StringArray>		
['Health and beauty', 'Electronic accessories', 'Home and lifestyle', 'Sports and travel', 'Food and beverages', 'Fashion accessories']		
Length: 6, dtype: str		
Uncertain Outcomes:		
['High Revenue', 'Low Revenue']		

Aashu Yadav
T050

16. Construct a decision tree in which each product line represents a decision alternative

and High Revenue and Low Revenue represent uncertain outcomes.

17. Display the probability and payoff at every outcome of the decision tree.

Code:

```
print("\n===== DECISION TREE  
=====")  
for product in decision_alternatives:  
    print("\n", product)  
    product_data = decision_df[  
        decision_df["Product Line"] == product  
    ]  
    for _, row in product_data.iterrows():  
        print(  
            " |——",  
            row["Revenue Level"],  
            "| Probability =",  
            round(row["Probability"], 3),  
            "| Payoff =",  
            round(row["Payoff"], 2)  
        )
```

Output:

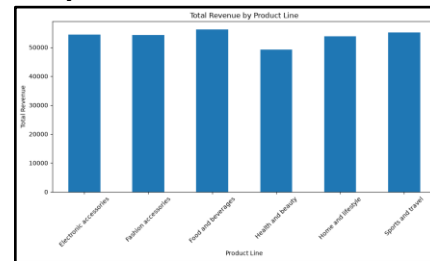
```
===== DECISION TREE =====  
  
Health and beauty  
|—— High Revenue | Probability = 0.533 | Payoff = 497.22  
|—— Low Revenue | Probability = 0.467 | Payoff = 125.62  
  
Electronic accessories  
|—— High Revenue | Probability = 0.465 | Payoff = 537.01  
|—— Low Revenue | Probability = 0.535 | Payoff = 130.92  
  
Home and lifestyle  
|—— High Revenue | Probability = 0.512 | Payoff = 529.58  
|—— Low Revenue | Probability = 0.487 | Payoff = 133.8  
  
Sports and travel  
|—— High Revenue | Probability = 0.53 | Payoff = 518.55  
|—— Low Revenue | Probability = 0.47 | Payoff = 121.68  
  
Food and beverages  
|—— High Revenue | Probability = 0.494 | Payoff = 515.13  
|—— Low Revenue | Probability = 0.506 | Payoff = 134.59  
  
Fashion accessories  
|—— High Revenue | Probability = 0.472 | Payoff = 512.33  
|—— Low Revenue | Probability = 0.528 | Payoff = 119.9
```

18. Create a visualization showing the total revenue of each product line.

Code:

```
plt.figure(figsize=(10, 6))  
total_revenue.plot(kind="bar")  
plt.title("Total Revenue by Product Line")  
plt.xlabel("Product Line")  
plt.ylabel("Total Revenue")  
plt.xticks(rotation=45)  
plt.tight_layout()  
plt.show()
```

Output:

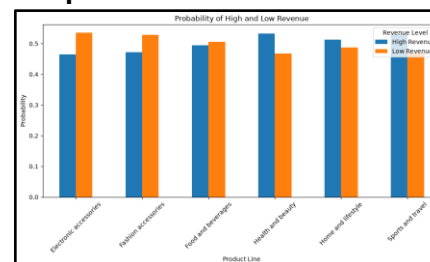


19. Create a visualization showing the probability of High Revenue and Low Revenue for each product line.

Code:

```
revenue_probability.plot(  
    kind="bar",  
    figsize=(10, 6)  
)  
plt.title("Probability of High and Low Revenue")  
plt.xlabel("Product Line")  
plt.ylabel("Probability")  
plt.xticks(rotation=45)  
plt.legend(title="Revenue Level")  
plt.tight_layout()  
plt.show()
```

Output:



20. Explain which product line has the highest probability of achieving High Revenue.

Code:

```
high_probability = revenue_probability["High  
Revenue"]  
highest_probability_product =  
high_probability.idxmax()  
print(  
    "\nProduct Line with Highest Probability of High  
Revenue:",  
    highest_probability_product  
)  
print(  
    "Probability:",  
    high_probability.max())
```

Aashu Yadav
T050

)

Output:

```
Product Line with Highest Probability of High Revenue: Health and beauty
Probability: 0.5328947368421053
```

Part B: Apply Expected Value

21. Explain how Expected Value can be used to select the best product line for a business

Decision.

Code:

```
print("\nExpected Value helps us select the product line")
print("that provides the highest expected payoff.")
```

Output:

```
Expected Value helps us select the product line
that provides the highest expected payoff.
```

22. Write the mathematical formula for calculating Expected Value.

Code:

```
print("\nExpected Value Formula:")
print("EV = Sum of (Probability × Payoff)")
```

Output:

```
Expected Value Formula:
EV = Sum of (Probability × Payoff)
```

23. Calculate the Expected Value of one product line using its outcome probabilities and Payoffs.

Code:

```
one_product = decision_alternatives[0]
one_product_data = decision_df[
    decision_df["Product Line"] == one_product
]
expected_value_one = (
    one_product_data["Probability"] *
    one_product_data["Payoff"]
).sum()
print(
    "\nExpected Value of",
    one_product,
    "=",
    expected_value_one
)
```

Output:

```
Expected Value of Health and beauty = 323.6430197368422
```

24. Calculate the Expected Value for every product line.

Code:

```
expected_values = (
    decision_df.assign(
        Expected_Value=
        decision_df["Probability"] *
        decision_df["Payoff"]
    )
    .groupby("Product Line")["Expected_Value"]
    .sum()
)
print("\nExpected Value of Every Product Line:")
print(expected_values)
```

Output:

```
Expected Value of Every Product Line:
Product Line
Electronic accessories    319.632538
Fashion accessories       305.089298
Food and beverages        322.671517
Health and beauty         323.643020
Home and lifestyle        336.636956
Sports and travel         332.065220
Name: Expected_Value, dtype: float64
```

25. Create a DataFrame containing Product Line, High Revenue Probability, High Revenue Payoff, Low Revenue Probability, Low Revenue Payoff, and Expected Value.

Code:

```
ev_data = []
for product in decision_alternatives:
    high_prob = revenue_probability.loc[product].get(
        "High Revenue", 0
    )
    low_prob = revenue_probability.loc[product].get(
        "Low Revenue", 0
    )
    high_payoff = average_payoff.get(
        (product, "High Revenue"), 0
    )
    low_payoff = average_payoff.get(
        (product, "Low Revenue"), 0
    )
    ev = (
        high_prob * high_payoff
        +
        low_prob * low_payoff
    )
    ev_data.append([
        product,
        high_prob,
        high_payoff,
        low_prob,
        low_payoff,
        ev
    ])
```

Aashu Yadav
T050

```
        low_payoff,
        ev
    ])
ev_df = pd.DataFrame(
    ev_data,
    columns=[
        "Product Line",
        "High Revenue Probability",
        "High Revenue Payoff",
        "Low Revenue Probability",
        "Low Revenue Payoff",
        "Expected Value"
    ]
)
print("\nExpected Value DataFrame:")
print(ev_df)
```

Output:

Expected Value DataFrame:				
	Product Line	High Revenue Probability	High Revenue Payoff	Expected Value
0	Health and beauty	0.532895	497	
1	Electronic accessories	0.464706	537	
2	Home and lifestyle	0.512500	529	
3	Sports and travel	0.530120	518	
4	Food and beverages	0.494253	515	
5	Fashion accessories	0.471910	512	
	Low Revenue Probability	Low Revenue Payoff	Expected Value	
0	0.467105	125.616972	323.643020	
1	0.535294	130.923577	319.632538	
2	0.487500	133.796519	336.636956	
3	0.469880	121.676154	332.065220	
4	0.505747	134.589358	322.671517	
5	0.528090	119.896037	305.089298	

26. Compare the Expected Values of all product lines.

Code:

```
print("\nExpected Value Comparison:")
print(
    ev_df[
        ["Product Line", "Expected Value"]
    ].sort_values(
        by="Expected Value",
        ascending=False
    )
)
```

Output:

Expected Value Comparison:		
	Product Line	Expected Value
2	Home and lifestyle	336.636956
3	Sports and travel	332.065220
0	Health and beauty	323.643020
4	Food and beverages	322.671517
1	Electronic accessories	319.632538
5	Fashion accessories	305.089298

27. Identify the product line having the highest Expected Value.

Code:

```
highest_ev_product = ev_df.loc[
    ev_df["Expected Value"].idxmax(),
```

```
"Product Line"
]
highest_ev = ev_df["Expected Value"].max()
print("\nHighest Expected Value Product Line:")
print(highest_ev_product)
print("Expected Value:", highest_ev)
```

Output:

```
Highest Expected Value Product Line:
Home and lifestyle
Expected Value: 336.63695625
```

28. Identify the product line having the lowest Expected Value.

Code:

```
lowest_ev_product = ev_df.loc[
    ev_df["Expected Value"].idxmin(),
    "Product Line"
]
lowest_ev = ev_df["Expected Value"].min()
print("\nLowest Expected Value Product Line:")
print(lowest_ev_product)
print("Expected Value:", lowest_ev)
```

Output:

```
Lowest Expected Value Product Line:
Fashion accessories
Expected Value: 305.089297752809
```

29. Rank all product lines from highest to lowest Expected Value.

Code:

```
ranking = ev_df[
    ["Product Line", "Expected Value"]
].sort_values(
    by="Expected Value",
    ascending=False
)
ranking["Rank"] = range(1, len(ranking) + 1)
print("\nProduct Line Ranking:")
print(ranking)
```

Output:

Product Line Ranking:			
	Product Line	Expected Value	Rank
2	Home and lifestyle	336.636956	1
3	Sports and travel	332.065220	2
0	Health and beauty	323.643020	3
4	Food and beverages	322.671517	4
1	Electronic accessories	319.632538	5
5	Fashion accessories	305.089298	6

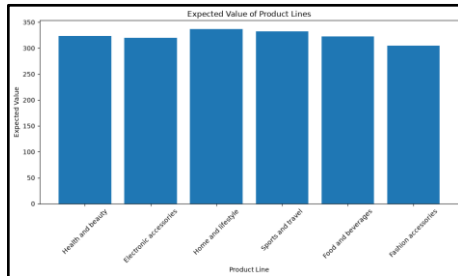
30. Create a bar chart comparing the Expected Values of all product lines.

Code:

```
plt.figure(figsize=(10, 6))
plt.bar(
    ev_df["Product Line"],
    ev_df["Expected Value"]
```

```
)  
plt.title("Expected Value of Product Lines")  
plt.xlabel("Product Line")  
plt.ylabel("Expected Value")  
plt.xticks(rotation=45)  
plt.tight_layout()  
plt.show()
```

Output:



31. Compare the product line with the highest probability of High Revenue with the product line having the highest Expected Value.

Code:

```
print(  
    "\nHighest Probability of High Revenue:",  
    highest_probability_product  
)  
print(  
    "Highest Expected Value:",  
    highest_ev_product)
```

Output:

```
Highest Probability of High Revenue: Health and beauty  
Highest Expected Value: Home and lifestyle
```

32. Determine whether the product line with the highest probability of High Revenue is also the product line with the highest Expected Value.

Code:

```
if highest_probability_product == highest_ev_product:  
    print(  
        "\nYes, both are the same product line."  
    )  
else:  
    print(  
        "\nNo, they are different product lines."  
    )
```

Output:

```
No, they are different product lines.
```

33. Change the probability of one possible outcome and recalculate the Expected Value

to observe how the decision changes.

Code:

```
modified_high_probability = 0.70  
modified_low_probability = 0.30  
product = one_product  
high_payoff = average_payoff.get(  
    (product, "High Revenue"), 0  
)  
low_payoff = average_payoff.get(  
    (product, "Low Revenue"), 0  
)  
modified_ev_probability = (  
    modified_high_probability * high_payoff  
    +  
    modified_low_probability * low_payoff  
)  
print("\nModified Probability:")  
print("High Revenue Probability =",  
    modified_high_probability)  
print("Low Revenue Probability =",  
    modified_low_probability)  
print("New Expected Value =",  
    modified_ev_probability)
```

Output:

```
Modified Probability:  
High Revenue Probability = 0.7  
Low Revenue Probability = 0.3  
New Expected Value = 385.7400767344809
```

34. Change the payoff of one possible outcome and recalculate the Expected Value to observe how the decision changes.

Code:

```
modified_high_payoff = high_payoff * 1.10  
modified_ev_payoff = (  
    high_probability.loc[product] *  
    modified_high_payoff  
    +  
    revenue_probability.loc[product].get("Low  
Revenue", 0)  
    * low_payoff  
)  
print("\nModified High Revenue Payoff:")  
print(modified_high_payoff)  
print("New Expected Value:", modified_ev_payoff)
```

Output:

Aashu Yadav
T050

```
Modified High Revenue Payoff:
546.9435481481482
New Expected Value: 350.1396868421054
```

35. Develop a Python program that automatically calculates the Expected Value of every product line and selects the product line with the highest Expected Value.

Code:

```
best_product = ev_df.loc[
    ev_df["Expected Value"].idxmax()
]
print("\n===== AUTOMATIC DECISION
=====")
print(
    "Best Product Line:",
    best_product["Product Line"]
)
print(
    "Highest Expected Value:",
    best_product["Expected Value"]
)
```

Output:

```
===== AUTOMATIC DECISION =====
Best Product Line: Home and lifestyle
Highest Expected Value: 336.63695625
```

36. Display the final decision tree along with the Expected Value of each product line.

Code:

```
print("\n===== FINAL DECISION TREE
=====")
for product in decision_alternatives:
    product_ev = ev_df.loc[
        ev_df["Product Line"] == product,
        "Expected Value"
    ].iloc[0]
    print("\n", product)
    print(" Expected Value =", round(product_ev, 2))
    product_data = decision_df[
        decision_df["Product Line"] == product
    ]
    for _, row in product_data.iterrows():
```

```
print(
    " |—",
    row["Revenue Level"],
    "→ Probability:",
    round(row["Probability"], 3),
    "Payoff:",
    round(row["Payoff"], 2)
)
```

Output:

```
===== FINAL DECISION TREE =====
Health and beauty
Expected Value = 323.64
|— High Revenue → Probability: 0.533 Payoff: 497.22
|— Low Revenue → Probability: 0.467 Payoff: 125.62

Electronic accessories
Expected Value = 319.63
|— High Revenue → Probability: 0.465 Payoff: 537.01
|— Low Revenue → Probability: 0.535 Payoff: 130.92

Home and lifestyle
Expected Value = 336.64
|— High Revenue → Probability: 0.512 Payoff: 529.58
|— Low Revenue → Probability: 0.487 Payoff: 133.8

Sports and travel
Expected Value = 332.07
|— High Revenue → Probability: 0.53 Payoff: 518.55
|— Low Revenue → Probability: 0.47 Payoff: 121.68

Food and beverages
Expected Value = 322.67
...
Fashion accessories
Expected Value = 305.09
|— High Revenue → Probability: 0.472 Payoff: 512.33
|— Low Revenue → Probability: 0.528 Payoff: 119.9
Output is truncated. View as a scrollable element or open in a text editor. Adjust co...
```

37. Write a business recommendation identifying the product line that should be selected based on the Expected Value.

Code:

```
print("\n===== BUSINESS
RECOMMENDATION =====")
print(
    "The business should select",
    highest_ev_product,
    "because it has the highest Expected Value of",
    round(highest_ev, 2)
)
print(
    "This means it provides the best expected financial
outcome"
    " among all product lines."
)
```

Output:

```
===== BUSINESS RECOMMENDATION =====
The business should select Home and lifestyle because it has the
This means it provides the best expected financial outcome among
```